

Углубленная теория по ИТиП (Python): Структура, Алгоритмы и Принципы

Блок 1. Архитектура, Запуск и Среда (Вопросы 1–2)

1.1. Как Python выполняет код (Алгоритм запуска)

Python — это не просто интерпретатор, читающий текст строка за строкой. Это двухэтапная система:

1. **Компиляция в байт-код:** Исходный код (`.py`) транслируется в промежуточный язык — байт-код (`.pyc`). Байт-код — это платформонезависимые инструкции для виртуальной машины.
 - *Зачем:* Чтобы не парсить текст каждый раз. Если файл не менялся, Python берет готовый `.pyc` из папки `__pycache__`.
2. **Выполнение PVM (Python Virtual Machine):** Виртуальная машина читает байт-код и выполняет его на процессоре. Она управляет памятью, типами и исключениями.

1.2. Среды разработки (IDE) и запуск

- **IDLE:** Базовая среда. Главное отличие — наличие **Интерактивной оболочки** (Shell), где код выполняется мгновенно после ввода, и **Редактора**, где код сохраняется в файл.
 - *Алгоритм работы IDLE:* При нажатии `F5` он сохраняет файл, компилирует его и передает байт-код интерпретатору, выводя результат в Shell.
- **Другие способы запуска:**
 - **Embedding (Встраивание):** Python можно встроить в программу на C/C++. Основная программа вызывает `Py_Initialize()`, загружает интерпретатор как библиотеку и выполняет скрипты. Это используется для создания плагинов или макросов в больших приложениях.
 - **Freezing (Заморозка):** Инструменты вроде `PyInstaller` берут твой скрипт, сам интерпретатор Python и все библиотеки, упаковывают их в один `.exe` файл. При запуске этот файл распаковывает всё во временную папку и запускает встроенный интерпретатор. Пользователю не нужно устанавливать Python.

Блок 2. Модель данных: Объекты, Ссылки и Типы (Вопросы 3, 14, 21, 26–27)

Это фундамент всего языка. Если ты поймешь этот блок, остальное станет очевидным.

2.1. Всё есть объект

В Python нет примитивных типов (как `int` в Java). Число `5` — это полноценный объект в куче (heap) памяти. У каждого объекта есть три свойства:

1. **Тип (Type):** Определяет, какие операции допустимы (число можно умножать, строку — нет). Тип неизменяем.
2. **Значение (Value):** Сами данные.
3. **Идентификатор (ID):** Уникальный адрес в памяти (`id(obj)`).

2.2. Переменные — это ссылки (Labels), а не коробки

- **Алгоритм присваивания (`x = 5`):**
 1. Создается объект `5` в памяти (если его еще нет).
 2. Создается имя `x`.
 3. Имя `x` привязывается (ссылается) на объект `5`.
- **Динамическая типизация:** Тип хранится в объекте, а не в переменной. Поэтому `x = 5`, а затем `x = "text"` — это не ошибка. Мы просто отвязали ярлык `x` от числа и привязали к строке.

2.3. Изменяемость (Mutability) и Разделяемые ссылки

Это главная причина багов у новичков.

- **Неизменяемые (Immutable):** `int`, `float`, `str`, `tuple`. Их значение нельзя поменять. Любая операция «изменения» создает **новый** объект.

- **Изменяемые (Mutable):** `list`, `dict`, `set`. Их можно менять «на месте» (in-place). ID объекта остается прежним.

Ловушка разделяемых ссылок:

```
a = [1, 2]
b = a      # b НЕ копирует список! b ссылается на тот же объект в памяти.
b.append(3) # Мы меняем объект в памяти.
print(a)   # Вывод: [1, 2, 3]. a тоже изменился, потому что это один объект.
```

Решение: Для копирования используйте `b = a.copy()` или `b = a[:]`.

2.4. Сборка мусора (Garbage Collection)

Python автоматически очищает память.

- **Основной механизм:** Подсчет ссылок (Reference Counting). У каждого объекта есть счетчик: сколько переменных на него ссылается. Когда счетчик становится 0, объект удаляется.
- **Дополнительный механизм:** Циклический сборщик (Cyclic GC). Удаляет объекты, которые ссылаются друг на друга, но недоступны из кода (циклические ссылки).

Блок 3. Основные структуры данных: Устройство и Алгоритмы (Вопросы 4–8, 22–24)

3.1. Строки (`str`) — Неизменяемая последовательность Unicode

- **Unicode:** В Python 3 строки хранят символы, а не байты. Каждый символ имеет уникальный код.
- **Внутреннее устройство (PEP 393):** Python экономит память. Если в строке только ASCII (английские буквы), он тратит 1 байт на символ. Если есть кириллица — 2 байта. Если эмодзи — 4 байта. Это происходит автоматически.
- **Неизменяемость:** Методы строк (`.upper()`, `.replace()`) никогда не меняют исходную строку. Они создают и возвращают новую.

```
s = "hello"
s.upper()    # Эта строка ничего не делает с s, она создает новую строку и тут же выбрасывает её.
s = s.upper() # Правильно: мы привязываем имя s к новой строке.
```

3.2. Списки (`list`) — Динамический массив ссылок

- **Внутреннее устройство:** Список хранит не сами элементы, а **ссылки** на объекты в других местах памяти.
- **Алгоритм роста:** Когда список заполняется, Python выделяет больше памяти, чем нужно (с запасом), чтобы операция `.append()` работала быстро (в среднем за $O(1)$).
- **Срезы (`s[start:stop:step]`):** Возвращают **новый** список. Это поверхностная копия.

3.3. Словари (`dict`) — Хеш-таблица

- **Алгоритм работы:** Словарь не перебирает элементы при поиске. Он берет ключ, вычисляет его **хеш** (число) и сразу идет по этому адресу в памяти.
- **Сложность:** Поиск, добавление и удаление работают за $O(1)$ — мгновенно, независимо от размера словаря.
- **Требование к ключам:** Ключ должен быть хешируемым (неизменяемым). Поэтому ключом может быть `str` или `tuple`, но не `list`.
- **Порядок:** С Python 3.7+ словари сохраняют порядок вставки элементов.

3.4. Кортежи (`tuple`) — Неизменяемый список

- **Зачем нужны:**
 1. **Защита данных:** Гарантируют, что набор данных не изменится.
 2. **Ключи словарей:** Так как они неизменяемы, их можно использовать как ключи.
 3. **Производительность:** Занимают меньше памяти и создаются быстрее списков, так как Python знает их размер заранее и не резервирует место под рост.

Блок 4. Операции, Выражения и Числа (Вопросы 9–13, 31–32)

4.1. Деление в Python 3

- `/` (Истинное деление): Всегда возвращает `float`. `5 / 2 = 2.5`.
- `//` (Целочисленное деление): Округляет **вниз** (к минус бесконечности).
 - `5 // 2 = 2`
 - `-5 // 2 = -3` (Важно! Не к нулю, а вниз).

4.2. Побитовые операции

Работают с двоичным представлением чисел.

- `&` (AND): 1 только если оба бита 1.
- `|` (OR): 1 если хотя бы один бит 1.
- `^` (XOR): 1 если биты разные.
- `<<` (Сдвиг влево): Умножение на 2 в степени сдвига. `5 << 1` равно `10`.
- `>>` (Сдвиг вправо): Деление на 2 в степени сдвига.

4.3. Операторы присваивания

- `a += b`: Для изменяемых типов (списков) это изменение **на месте** (быстро, ID не меняется). Для неизменяемых (чисел) — создание нового объекта.
- **Распаковка**: `a, b = [1, 2]`. Python сопоставляет элементы слева и справа.
- **Звездочка** `*`: `first, *middle, last = [1, 2, 3, 4, 5]`. `middle` станет списком `[2, 3, 4]`.

Блок 5. Управление потоком: Условия и Циклы (Вопросы 28–30, 34–36)

5.1. Логика `if` и истинность (Truthiness)

Python проверяет на истинность любой объект.

- **False (Ложь)**: `0`, `0.0`, `""` (пустая строка), `[]` (пустой список), `None`, `False`.
- **True (Истина)**: Всё остальное. Даже `-1` или `" "` (строка с пробелом) — это True.
- **Тернарный оператор**: `x = "Да" if condition else "Нет"`. Это выражение, оно возвращает значение.

5.2. Цикл `while` и `else`

Уникальная фишка Python: у цикла есть блок `else`.

- **Алгоритм**: `else` выполняется только если цикл завершился **естественным путем** (условие стало False).
- **Если был `break`**: Блок `else` **пропускается**.
- **Применение**: Идеально для поиска. Если нашли элемент — делаем `break` (и `else` не сработает). Если перебрали всё и не нашли — сработает `else`.

5.3. Цикл `for` и Протокол итерации

Цикл `for` в Python — это не счетчик, а итератор.

- **Алгоритм работы `for x in lst`**:
 1. Вызывает `iter(lst)` -> получает итератор.
 2. В бесконечном цикле вызывает `next(iterator)`.
 3. Присваивает значение `x` и выполняет тело цикла.
 4. Когда элементы кончатся, итератор выбрасывает исключение `StopIteration`.
 5. Цикл `for` ловит это исключение и тихо завершается.

- `range()` : В Python 3 это не список, а «ленивый» объект. Он не хранит числа в памяти, а вычисляет следующее число по формуле при каждом шаге. Экономит гигабайты памяти.

Блок 6. Функции: Область видимости и Аргументы (Вопросы 42–53, 60)

6.1. Правило LEGB (Поиск имен)

Когда функция использует переменную, Python ищет её в порядке:

1. **Local** (Локально в функции).
2. **Enclosing** (Во внешней функции, если есть вложенность).
3. **Global** (Глобально в файле).
4. **Built-in** (Встроенные имена, типа `print`).

6.2. Замыкания (Closures) и `nonlocal`

- **Замыкание:** Внутренняя функция запоминает переменные из внешней функции даже после её завершения.
- **Проблема:** В Python 3 внутренняя функция может *читать* внешнюю переменную, но не может её *перезаписать* (присвоить новое значение), так как Python сочтет её локальной.
- **Решение:** Ключевое слово `nonlocal var`. Оно говорит: «Не создавай новую локальную переменную, используй ту, что во внешней функции».

6.3. Передача аргументов

- **Pass-by-assignment:** Аргументы передаются как ссылки.
- **Ловушка изменяемых аргументов по умолчанию:**

```
def func(a=[]): # ОШИБКА! Список создается один раз при определении функции.
    a.append(1)
    return a

# При повторных вызовах список будет расти: [1], [1, 1], [1, 1, 1]...
```

Решение: Использовать `None` как значение по умолчанию и создавать список внутри: `if a is None: a = []`.

6.4. `*args` и `**kwargs`

- `*args` : Собирает все лишние позиционные аргументы в **кортеж**.
- `**kwargs` : Собирает все лишние именованные аргументы в **словарь**.
- **Keyword-only arguments:** Если в сигнатуре функции стоит голая звездочка `def func(a, *, b):`, то аргумент `b` можно передать **только** по имени (`func(1, b=2)`). Это защищает от ошибок порядка аргументов.

Блок 7. Модули и Пакеты (Вопросы 61–77)

7.1. Алгоритм импорта (`import`)

1. **Поиск:** Python ищет файл в `sys.path` (текущая папка -> PYTHONPATH -> стандартная библиотека).
2. **Компиляция:** Если нашел `.py`, компилирует в `.pyc` (если нужно).
3. **Выполнение:** Выполняет код файла сверху вниз. Все созданные имена становятся атрибутами объекта модуля.
4. **Кэширование:** Модуль загружается в память **один раз**. Повторный `import` берет объект из кэша `sys.modules`, код не выполняется снова.

7.2. `if __name__ == "__main__":`

- При прямом запуске файла переменная `__name__` равна `"__main__"`.
- При импорте файла в другой скрипт `__name__` равна имени файла.
- **Зачем:** Позволяет писать тесты или демо-код внутри модуля, который выполнится только при прямом запуске, но не при импорте.

7.3. Пакеты и `__init__.py`

- **Пакет:** Папка с модулями. Файл `__init__.py` превращает папку в пакет и выполняется при первом импорте любого модуля из этой папки.
- **Относительный импорт:** `from . import module` (точка означает «текущая папка»). Работает только внутри пакетов.
- **Namespace Packages (Python 3.3+):** Папки без `__init__.py` могут быть частью одного пакета, если они лежат в разных местах `sys.path`. Python объединяет их.

Блок 8. Объектно-Ориентированное Программирование (ООП) (Вопросы 78–82)

8.1. Три кита ООП в Python

1. Инкапсуляция (Скрытие данных)

В Python нет жесткого `private`. Есть соглашения:

- `_var` (одно подчеркивание): «Это внутреннее, не трогай». Интерпретатор не запрещает доступ, но IDE предупреждает. При `from module import *` такие имена не импортируются.
- `__var` (два подчеркивания): Name Mangling (искажение имен). Python меняет имя на `_ClassName__var`. Это защищает от случайного переопределения в дочерних классах.

2. Наследование и MRO (Method Resolution Order)

- Python поддерживает множественное наследование.
- **Алгоритм поиска метода (MRO):** Python использует алгоритм C3-линеаризации. Он строит линейный порядок классов, чтобы каждый родитель встречался только один раз и соблюдался порядок объявления.
- Посмотреть порядок: `Class.mro()`.
- `super()`: Вызывает метод следующего класса в цепочке MRO, а не просто родителя. Это критически важно при множественном наследовании, чтобы не вызвать метод дважды.

3. Полиморфизм (Утиная типизация)

Python не проверяет тип объекта строго. Ему важно наличие методов.

- *Суть:* «Если это ходит как утка и крикает как утка, то для моей функции это утка».
- Ты можешь написать функцию `process(obj)`, которая вызывает `obj.save()`. Ей всё равно, является ли `obj` Пользователем или Заказом.

8.2. Перегрузка операций (Магические методы)

Ты можешь научить свои объекты реагировать на операторы `+`, `==`, `[]`.

- `__add__(self, other)`: для оператора `+`.
- `__eq__(self, other)`: для оператора `==`. По умолчанию `==` сравнивает ID объектов. Перегрузив его, ты можешь сравнивать по содержимому.
- `__str__(self)`: для `print()`. Возвращает человекочитаемое представление.
- `__repr__(self)`: для отладки. Возвращает однозначное представление (часто код, который воссоздаст объект).

Блок 9. Исключения и Контекстные менеджеры (Вопросы 83–89)

9.1. Структура `try/except/else/finally`

- `try`: Код, где может быть ошибка.
- `except`: Обработка конкретной ошибки.
- `else`: Выполняется, **только если в try не было ошибок**. Позволяет отделить защищаемый код от остальной логики.

- `finally` : Выполняется **ВСЕГДА**, даже если был `return` или необработанная ошибка. Используется для закрытия файлов/соединений.

9.2. `raise` и сцепление исключений

- `raise` : Генерирует ошибку вручную.
- `raise ... from ...` : Явно связывает две ошибки. Показывает в трассировке: «Ошибка Б произошла из-за Ошибки А». Помогает отладке.

9.3. Контекстные менеджеры (`with`)

- **Протокол**: Объект должен иметь методы `__enter__` (вход) и `__exit__` (выход).
 - **Алгоритм**:
 1. Вызывается `__enter__`, результат пишется в переменную после `as`.
 2. Выполняется блок кода.
 3. Даже если была ошибка, вызывается `__exit__`. Если `__exit__` возвращает `True`, ошибка подавляется. Если `False / None` — ошибка идет дальше.
 - **Зачем**: Гарантирует очистку ресурсов (закрытие файлов) без написания громоздкого `try/finally`.
-

Блок 10. Продвинутые темы: Генераторы, Декораторы, Байты (Вопросы 54–59, 90–97)

10.1. Генераторы (`yield`)

- **Суть**: Функция с `yield` не выполняет код сразу. Она возвращает объект-генератор.
- **Алгоритм**: При вызове `next()` функция выполняется до первого `yield`, отдает значение и **замораживает** свое состояние (все локальные переменные сохраняются). При следующем `next()` продолжает с того же места.
- **Экономия памяти**: Генератор не хранит все данные в памяти, а выдает их по одному. Идеально для обработки больших файлов.

10.2. Строки vs Байты в Python 3

- **`str (Unicode)`**: Текст. Неизменяем. Хранит символы.
 - **`bytes`** : Сырые байты (0-255). Неизменяем. Хранит числа.
 - **`bytearray`** : Изменяемые байты.
 - **Главное правило**: Нельзя смешивать `str` и `bytes`.
 - Текст -> Байты: `.encode('utf-8')`
 - Байты -> Текст: `.decode('utf-8')`
 - **Файлы**:
 - Текстовый режим (`'r'`): Автоматически декодирует байты в `str`.
 - Бинарный режим (`'rb'`): Читает сырые `bytes`. Никакого декодирования.
-

Шпаргалка по версиям для экзамена:

1. **Python 2.7**: `print` — оператор, `str` — байты, деление `/` — целочисленное.
2. **Python 3.8**: `print()` — функция, `str` — Unicode, деление `/` — истинное. Появился оператор моржа `:=`.
3. **Python 3.12+**: Ускорение работы (Faster CPython), улучшенные сообщения об ошибках, удаление устаревших модулей.